Lab 4: Gaps and merge

Today we looked for missing values, decided what to do with them, and put two tables together with `merge`. The most important lesson was to ask **why** a value is missing, and to check the numbers after every step. In this lab you try it all on **new data**: the loans and the members of a library. **The library and its data are made up**, but the problems are of the kinds that real data has. This lab has two parts, and **you choose**.

Part 1: Practise repeats the steps from today on the new tables. The problems are of the same kinds as today, in new places. Do this part if you want to try the whole routine once more on your own.

Part 2: Go deeper is for you who felt "I understood this, and I want to go further". The questions are more open, and you have to decide what is right when the data does not give a clear answer.

I do not expect you to do both parts. It is your own choice, and you are welcome to contact me if you want to talk about which part to take.

Download **library_loans.csv** and **library_members.csv** from Teams and save them in your `data` folder. Work in a new notebook. Keep the originals: load each table into `..._raw` and work on a copy. Write a short comment above each step, in your own words. If you get stuck: read the error message from the bottom up, print the thing you are working with, and look at the lecture notebook. Then ask.

Part 1: Practise

Look for gaps, decide what to do with them, and merge the two tables, with the same steps as today.

Tools you need

```
loans = pd.read_csv('data/library_loans.csv', parse_dates=['loan_date', 'return_date'])
members_raw = pd.read_csv('data/library_members.csv')       members = members_raw.copy()
table.isna().sum()      (table.isna().mean() * 100).round(1)      table['x'].notna()
table['x'].where(table['x'].between(1920, 2015))              # outside the range -> NaN
table.dropna()      table.dropna(subset=['x'])      table['x'].fillna('Unknown')
table['x'].median()      table['x'].std()
loans.merge(members, on='member_id', how='left', validate='many_to_one', indicator=True)
merged['_merge'].value_counts()           # both / left_only / right_only
table.drop_duplicates().reset_index(drop=True)      table.groupby('x')['y'].count()
```

Tasks

1 **Load the loans and look for gaps**
Load `library_loans.csv` with `parse_dates` for both date columns. Print the shape and the dtypes. Then count the missing values in every column, and show them as percent. Which two columns have gaps, and how many?

2 **Two kinds of missing**
Show the loans where `loan_date` is missing. Then look at `return_date`: how many are missing **and** the book is back (`returned` is `True`), and how many are missing and the book is **not** back? Which of the two kinds is "missing for a reason", and which is "missing by accident"? Create a column `has_return_date` with `notna()` and count it.

LEXICON

3 **Look at the members**

Load `library_members.csv`, make a copy, and print the shape. Run `info()` and `describe()` on `birth_year`. Something is wrong with the **minimum** and the **maximum**. Count how many members have each strange value, and check for duplicates in the whole row and in `member_id`. Write the list of problems as a comment.

4 **Placeholders and impossible values**

Nobody in this library is born in 1900, and nobody is born in the future. Use `where` and `between(1920, 2015)` to turn those values into missing values. How many birth years are missing now? Run `describe()` again and compare it with before.

5 **Remove, fill, or leave**

(a) Without saving the result, see how many rows are left with `dropna()`, with `dropna(subset=['branch'])` and with `dropna(subset=['birth_year'])`.
(b) Fill the missing `branch` with `'Unknown'` and count the branches.
(c) Try filling the missing `birth_year` with the median, **on the side**, in a separate variable. Compare the mean and the standard deviation before and after. What happens to the spread, and why? What do you decide to do with `birth_year`?

6 **Merge, and check**

Print the shape of both tables. Merge the loans with the members with `how='left'` and **without** `validate`. How many rows do you get, and why is it not 600? Calculate the total of `late_fee` before and after the merge. Then merge again with `validate='many_to_one'` and read the error from the bottom up.

On purpose: The merge with `validate` should give a `MergeError`.

7 **Fix the key, and find who is missing**

Remove the duplicates from `members` and merge again with `validate='many_to_one'` and `indicator=True`. Check the shape and the total of `late_fee`. How many loans have `left_only`, how many different members is that, and how many kr in late fees? What would an `inner` merge have given for the shape and the total?

8 **Use the merged table**

Give the loans without a member record the branch `'No member record'` (`fillna`). Count the loans per branch, biggest first. Which branch has the most loans? Write one sentence about the two groups `Unknown` and `No member record`: why are they not the same thing?

Expected

Task 1: (600, 8). `loan_date` has 3 missing (0.5 percent) and `return_date` has 36 (6.0 percent). The other columns have none.

Task 2: 3 rows with a missing `loan_date` (L-30463, L-30468 and L-30572). 30 loans have no return date and are not back (missing for a reason), and 6 have no return date but are back (missing by accident). `has_return_date`: True 564, False 36.

Task 3: (148, 3). `branch` has 136 values and `birth_year` has 130. `birth_year`: min 1900, max 2030, mean 1983.4. 8 members have 1900 and 3 have 2030. 4 duplicate rows and 4 duplicate `member_id`.

Task 4: 29 missing (18 + 8 + 3). `describe()`: count 119, mean 1987.82, min 1957, max 2010.

Task 5: (a) (107, 3), (136, 3) and (119, 3). (b) Central 40, South 34, North 32, East 17, Harbour 13, Unknown 12. (c) Median 1988.0. The mean is 1987.82 before and 1987.85 after, the standard deviation goes from 12.06 to 10.81.

Task 6: (600, 9) (the 8 original columns plus `has_return_date`) and (148, 3). The merge gives (616, 11): 16 extra rows, because 4 members are in the table twice. The total of `late_fee` goes from 7487.5 to 7725.0 (237.5 too much). The `validate` merge: `MergeError: Merge keys are not unique in right dataset; not a many-to-one merge`.

Task 7: after the fix (144, 3), and the merge gives (600, 12) with `indicator` (600, 11 without). Total 7487.5. `left_only` 31 loans, from 6 different members, 347.5 kr. With `inner`: (569, 11) and 7140.0.

Task 8: Central 144, South 130, North 126, East 72, Harbour 51, Unknown 46, No member record 31. `Unknown` is a member who exists but has no branch. `No member record` is a loan whose member is not in the table at all.

LEXICON

Part 2: Go deeper

Here you ask what the gaps mean, and you document your decisions. There are two new tools (`dt.year` and `how='outer'`), and they are in the box below.

Tools you need

```
merged['loan_date'].dt.year - merged['birth_year']          # age at the loan
loans.merge(members, on='member_id', how='outer', indicator=True)    # keep rows from BOTH tab
members['member_id'].isin(loans['member_id'])                       # is the member in the loa
table.groupby('x')['y'].agg(['count', 'mean'])
```

Tasks

1 **Who is missing, and does it matter?**
Look at the loans whose member is not in the member table. How many is that, what share of all loans, and what share of all late fees? Which genres do they borrow? Does it look like a pattern, or like chance? Write in a comment who you would **ask**, and what you would ask, to solve it.

2 **Members without loans**
Some members may never have borrowed a book. Find out with an `outer` merge and `indicator=True`: how many are `right_only`? Then check the same thing in another way, with `isin()`. Which of the two ways do you prefer, and why?

3 **How old are the readers?**
In your merged table, calculate the **age at the loan** with `loan_date.dt.year - birth_year`. How many loans have no age, and why? Then calculate the number of loans and the average age for every genre. Which genre has the youngest readers? Write what you **can** and **cannot** conclude.

4 **Write down your decisions**
In a markdown cell, make a list of every problem you found in the member table (missing `branch`, missing `birth_year`, the 1900 values, the 2030 values, the duplicates). For each, write what you did (**remove, fill or leave**), **why**, and what it does to the numbers. A colleague should be able to read your list and know exactly what you did.

5 **Your own question**
Come up with **one question** about the library that needs both tables. Write the question as a comment, answer it with code, and finish with a comment about what the answer does **not** tell you. That last part is the most important one.



Expected
Task 1: 31 loans (5.2 percent) and 347.5 kr (4.6 percent of 7487.5). The genres: Children 10, Sci-fi 5, Crime 4, Fantasy 4, Romance 4, History 4. There is no clear pattern: the number is small and the spread is about the same as for all loans. Good answers ask the person who keeps the member table why 6 members are missing.
Task 2: with `outer`: both 569, left_only 31, right_only 1. So only one member has never borrowed a book, and 143 of 144 members have borrowed.
Task 3: 143 loans have no age (for example the 31 loans without a member, loans of members with a missing `birth_year`, and loans without a `loan_date`). The mean age is 38.8 and there are 457 known ages. Per genre: History 35.2 (45 loans), Sci-fi 37.8 (66), Romance 38.1 (58), Crime 38.4 (120), Fantasy 39.8 (67), Children 41.1 (101). The differences are small and we do not know why the ages are missing.
Task 4: open. A good list has a decision, a reason and an effect for each of the five problems.

Stuck? Read the error message from the bottom up, print the data you are working with, and then ask.